VERDANT: A CPU-Deployable Machine Learning Platform for Plant Health Diagnosis, Crop Planning and Grounded Advice in Urban Vertical Farming

[Student Name 1]
Dept. of [Your Department]
School of Engineering
Anurag University
Hyderabad, India
[roll-number-1]@anurag.edu.in

[Student Name 2]
Dept. of [Your Department]
School of Engineering
Anurag University
Hyderabad, India
[roll-number-2]@anurag.edu.in

[Student Name 3]
Dept. of [Your Department]
School of Engineering
Anurag University
Hyderabad, India
[roll-number-3]@anurag.edu.in

[Guide Name]
[Designation], Dept. of [Your Department]
School of Engineering
Anurag University
Hyderabad, India
[guide-email]@anurag.edu.in

***Abstract*—Urban vertical farms can grow food close to consumers, but the small operators who run most of them rarely have access to plant pathologists, agronomists or the computing resources that commercial decision-support tools assume. This paper presents VERDANT, an open, CPU-deployable platform that integrates several machine-learning components behind a single web interface: two pretrained Vision Transformers for leaf-disease diagnosis and species identification, a retrieval-augmented advisor that grounds a Gemma 3 language model in a curated agronomy knowledge base, a physics-based agronomic engine for vapour-pressure-deficit and growth-index analysis, and four lightweight models trained from scratch for disease screening, crop recommendation, growth-cycle prediction and greenhouse climate forecasting. Every learned model is compared against classical baselines on held-out data, and the main design choices are probed with ablation and sensitivity studies. The custom models reach 83.6% accuracy (ROC AUC 0.917) for tomato late-blight screening, 87.9% for five-class crop recommendation, $R^2 = 0.856$ for lettuce growth prediction and $R^2 = 0.891$ (RMSE 0.98 °C) for one-hour-ahead temperature forecasting, a 34% error reduction over a persistence forecast. Dense MiniLM retrieval answers 79.4% of paraphrased grower queries correctly at rank one, more than twice the rate of TF-IDF and BM25, and indexing answers together with questions raises this to 92.6%. All components run on a laptop CPU with per-request latencies between 0.1 and 206 ms. The experiments also expose the limits of synthetic training data, which are discussed openly together with a roadmap for field validation.**

Keywords—urban farming, vertical farming, plant disease detection, vision transformer, convolutional neural network, random forest, retrieval-augmented generation, vapour pressure deficit, decision support system

I. Introduction

More than half of the world's population already lives in cities, and the United Nations projects that this share will reach about 68% by 2050 [1]. Feeding these cities is becoming harder as arable land shrinks, fresh water becomes scarce and food travels ever longer distances before it is eaten [2]. Vertical farming and other forms of controlled-environment agriculture (CEA) respond to this pressure by growing crops in stacked layers indoors, under artificial light and with tightly managed nutrients and climate [3], [4]. Because water is recirculated and space is used in three dimensions, such farms can produce considerably more food per square metre than open fields while using a fraction of the water [5].

Running a vertical farm, however, is knowledge-intensive. A grower must recognise the first symptoms of disease, choose crops that suit the available conditions, hold electrical conductivity (EC), pH, light and vapour pressure deficit (VPD) inside narrow bands, and anticipate climate swings before they stress the plants. Pathogens and pests alone remove an estimated 10–40% of the global yield of staple crops [6], and in a closed growing room a single undetected infection can spread through a whole rack within days. Commercial operators can afford agronomists and proprietary climate software; the hobbyists, start-ups, schools and community gardens that make up much of urban agriculture usually cannot.

Machine learning (ML) is widely studied as a way to close this expertise gap [7], [8]. Convolutional networks classify leaf diseases from photographs with high accuracy [9], [10], tree ensembles support crop and yield planning, Internet of Things (IoT) sensing makes continuous climate data cheap [11], [12], and retrieval-augmented language models can answer questions from a curated knowledge source [13]. In practice these capabilities are published as separate prototypes, are rarely compared with simple baselines, and often assume a GPU or a paid cloud service. A grower who wants to diagnose a leaf, pick a crop, estimate a harvest date and ask a follow-up question must stitch several tools together.

This paper presents VERDANT, a single platform that brings these functions together and evaluates each of them rigorously. Its main contributions are:

- **An integrated, CPU-only architecture** that serves vision, tabular, physiological and language components through one REST interface, with lazy

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

loading, concurrency control and graceful degradation when an external model is unavailable.

- **A grounded agronomic advisor** that combines dense MiniLM retrieval over a curated knowledge base with Gemma 3 generation, together with a 68-query paraphrase benchmark showing that dense retrieval more than doubles the rank-1 accuracy of TF-IDF and BM25.
- **A physics-based agronomic engine** that turns air temperature, humidity, light, EC and CO_2 into VPD, daily light integral (DLI) and an interpretable growth index based on Liebig's law of the minimum.
- **Four lightweight models trained from scratch**, each compared with three to six classical baselines and analysed with ablation, depth-sensitivity and feature-importance studies, all reproducible from a public training notebook and experiment scripts.
- **An honest account of what the evidence does and does not show**, including cases where simpler baselines win and the effect of synthetic training data.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the datasets, models and evaluation protocol. Section V reports and discusses the results, and Section VI concludes with directions for future work.

II. Related Work

A. Image-Based Plant Disease Detection

Deep learning has transformed image recognition since AlexNet won the ImageNet challenge [14]–[16]. The PlantVillage dataset, with more than 54,000 labelled leaf images of 14 crops and 26 diseases [17], made it possible to apply these networks to plant pathology. Mohanty *et al.* [9] reported 99.35% held-out accuracy with GoogLeNet, while noting a sharp drop on field images. Sladojevic *et al.* [18] recognised 13 disease types with an average precision of 96.3%, Ferentinos [10] reached 99.53% with VGG on 87,848 images of 25 species, Brahimi *et al.* [19] visualised the leaf regions that drive tomato-disease predictions, and Too *et al.* [20] found DenseNet fine-tuning both accurate and parameter-efficient. Residual networks [21] have since become a default backbone, and the Plant Pathology 2020 challenge [22] confirmed that photographs with complex backgrounds remain much harder than laboratory images.

The Transformer [23] has more recently been adapted to vision. The Vision Transformer (ViT) [24] treats an image as a sequence of patches processed by self-attention, and data-efficient training recipes have produced compact variants such as DeiT-Tiny with about 5.7 M parameters [25]. Borhani *et al.* [26] showed that ViT-based models suit automated plant-disease classification and analysed their accuracy–speed trade-off. VERDANT deploys public ViT checkpoints for diagnosis and, in parallel, trains a compact CNN from scratch to study what can be learned from very little data.

B. Crop Planning, Growth and Climate Prediction

For tabular agronomic data, classical ML remains competitive. In their review, Liakos *et al.* [7] found that tree ensembles and support vector machines dominate yield-prediction and crop-selection studies. Random Forests [27] are popular because they model non-linear interactions between soil nutrients and weather, resist overfitting through bootstrap aggregation, and expose feature importances that growers can interpret, while regularised linear models such as Ridge [28] provide transparent baselines for growth estimation. In greenhouse and plant-factory settings, Shamshiri *et al.* [29] surveyed climate-control strategies; recurrent networks such as LSTMs [30] have been used to forecast indoor temperature and humidity, and simple persistence and seasonal-naive forecasts remain the reference that any forecaster must beat [31].

C. Physiological Models in Controlled Environments

Plant growth in CEA is governed by well-understood physics. The saturation vapour pressure of air follows Tetens' empirical equation [32], and the resulting VPD controls transpiration, stomatal opening and nutrient uptake [33]. The DLI summarises the photosynthetically active photon dose a crop receives each day [4], and Liebig's law of the minimum states that growth is limited by the scarcest resource [34]. These relationships are rarely exposed to growers in an interactive, quantitative form; VERDANT encodes them in a lightweight engine that runs in the browser.

D. Retrieval and Language-Model Advisors

Pretrained language models, beginning with BERT [35], made natural-language question answering practical. Sentence-BERT [36] produces embeddings that can be compared with cosine similarity, and MiniLM [37] distils such models so that they run quickly on a CPU. Dense retrievers outperform sparse methods such as TF-IDF [38] and BM25 [39] when queries paraphrase the stored text [40]. Retrieval-augmented generation (RAG) [13] conditions a generator on retrieved passages, which curbs hallucination and allows knowledge to be updated without retraining, and open-weight models such as Gemma 3 [41] make the pattern affordable. Table I contrasts representative work with VERDANT: most prior systems address one task, whereas VERDANT combines vision, tabular prediction, physiological modelling and grounded advice in one deployable platform.

TABLE I. Capabilities of Representative Work Compared with VERDANT

Work	Dis.	Spe.	Crop	Clim.	Adv.	Base.
Mohanty *et al.* [9]	✓	–	–	–	–	✓
Ferentinos [10]	✓	–	–	–	–	✓
Too *et al.* [20]	✓	–	–	–	–	✓
Borhani *et al.* [26]	✓	–	–	–	–	✓
Lewis *et al.* [13]	–	–	–	–	✓	✓
VERDANT (ours)	✓	✓	✓	✓	✓	✓

Dis.: disease diagnosis; Spe.: species identification; Crop: crop or growth planning; Clim.: climate forecasting; Adv.: natural-language advice; Base.: comparison with baselines.

III. System Architecture

A. Design Goals

VERDANT was designed around four goals: (i) *accessibility*—every component must run on a commodity laptop without a GPU; (ii) *graceful degradation*—the platform must stay useful when the network or the external language model is unavailable; (iii) *modularity*—each model sits behind its own endpoint so that it can be replaced independently; and (iv) *reproducibility*—all custom models,

datasets and experiments can be regenerated from fixed random seeds.

B. Three-Tier Architecture

Fig. 1 shows the resulting three-tier design. The *presentation tier* is a single-page application written in React 19 and TypeScript, bundled with Vite and styled with Tailwind CSS. It hosts the Plant Doctor, Species Scanner, AI Advisor and Grow Planner, plus a set of client-side agronomic tools (growth simulator, VPD matrix, nutrient lab and dashboard) described in Section IV-E. The *application tier* is a FastAPI service exposing five REST endpoints. The *model tier* holds three pretrained Transformers downloaded from the Hugging Face Hub, the four models trained from scratch, the agronomy knowledge base with its cached embeddings, and an external Gemma 3 endpoint reached through the Ollama API.

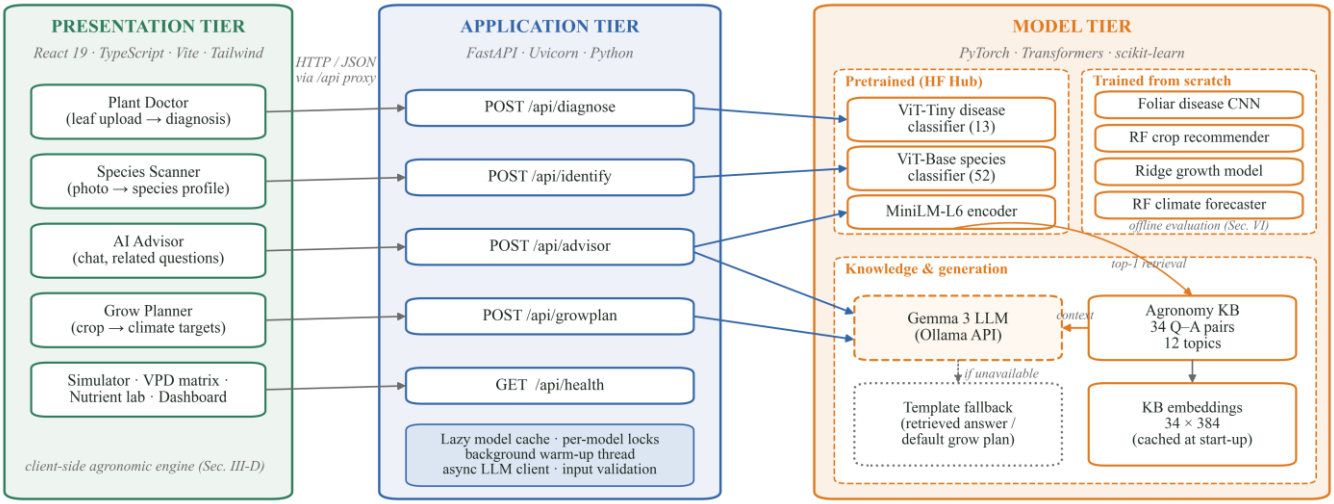


Fig. 1. Three-tier architecture of VERDANT. Solid arrows show request paths; dotted arrows show the fallback taken when the external language model is unavailable. The four models trained from scratch are evaluated offline in this study (Section V).

C. Application Tier and Concurrency

POST /api/diagnose and *POST /api/identify* accept a multipart image and return the five most probable classes; *POST /api/advisor* returns a grounded answer with its topic, retrieval score and related questions; *POST /api/growplan* returns structured climate targets for a crop; and *GET /api/health* reports which models are loaded. Models are loaded lazily into an in-memory cache guarded by one lock per model, so that concurrent first requests never load the same weights twice, and a background thread started from the application's lifespan hook warms the vision models and pre-computes the knowledge-base embeddings while the server already accepts traffic. Language-model calls are issued asynchronously with httpx so that a slow external response never blocks image inference. Invalid input yields HTTP 400 with a descriptive message, internal failures yield HTTP 500, and secrets are read from an environment file rather than source code.

IV. Methodology

A. Datasets and Preprocessing

The pretrained Transformers were used as released. For the four custom models, labelled data from real vertical farms were not available to the project, so four datasets were generated programmatically from value ranges and relationships taken from horticultural guidance [4]. Following common practice for synthetic training data [42], noise, missing values, corrupted files and class overlap were injected on purpose so that the models face realistic preprocessing and generalisation problems rather than trivially separable data. Every random process uses a fixed seed. Table II summarises the datasets and Fig. 2 visualises them.

Leaf images: A PlantVillage-style set [17] with two classes, tomato healthy and tomato late blight, was synthesised at 128 × 128 pixels with 320 images per class. The classes have overlapping colour ranges; lesions appear on only about 65% of diseased leaves to mimic early infections, and about 30% of healthy leaves carry harmless spots. Gaussian sensor noise ($\sigma = 34$ grey levels) and JPEG compression are applied. Three zero-byte files were detected and removed during cleaning, leaving 637 images split 80/20 into 509 training and 128 validation images.

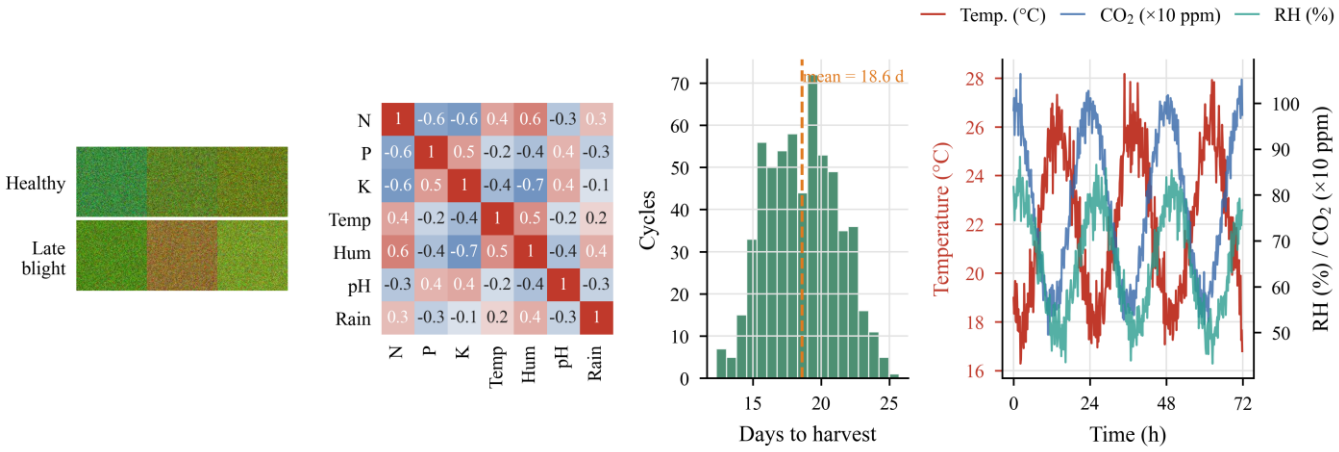
Crop recommendation: 1,200 samples with seven features—nitrogen (N), phosphorus (P), potassium (K), temperature, humidity, pH and rainfall—and five labels (rice, maize, chickpea, cotton and coffee). Gaussian measurement noise was added and 31, 27 and 26 values were removed from N, temperature and humidity; these were imputed with the class-wise median and outliers were capped at 1.5 times the interquartile range. A stratified split gives 960 training and 240 validation samples, standardised with training statistics only.

Lettuce growth: 600 hydroponic cycles described by DLI (11–19 mol m^{-2} d^{-1}), air temperature (18–24 °C), EC (0.9–2.1 mS/cm), pH (5.5–6.5) and water temperature (17–22 °C). Days to harvest fall with light, temperature and EC and carry Gaussian noise of $\sigma = 1.05$ days (range 12.3–25.7, mean 18.6). Note that pH and water temperature were deliberately given no effect, which provides a check on whether a model attributes importance correctly.

Greenhouse sensors: ten days of temperature, humidity and CO_2 at 10-minute intervals (1,441 steps) with diurnal cycles and sensor noise. Lag features were built for temperature at $t-1$, $t-2$ and $t-6$ and for humidity and CO_2 at $t-1$; the target is temperature one hour ahead ($t+6$). The series is split chronologically without shuffling into 1,143 training and 286 validation steps, so that no future information leaks into training.

TABLE II. DATASETS USED TO TRAIN AND EVALUATE THE CUSTOM MODELS

Dataset	Train / Val	Inputs	Target	Injected defects
Leaf images	509 / 128	128×128 RGB	2 classes	Noise, overlap, 3 corrupt files
Crop recommendation	960 / 240	7 numeric	5 crops	Noise, 84 missing values
Lettuce growth	480 / 120	5 numeric	Days	Noise, 2 irrelevant inputs
Greenhouse sensors	1,143 / 286	8 incl. lags	T at +1 h	Noise, temporal order



(a) Leaf image samples (b) Crop feature correlation (c) Lettuce growth cycles (d) Greenhouse sensor streams

Fig. 2. Overview of the four datasets: (a) validation leaf images, (b) Pearson correlation of the crop-recommendation features after cleaning, (c) distribution of lettuce cultivation times, and (d) the first three days of the greenhouse sensor streams.

B. *Pretrained Vision Transformer Classifiers*

Both image endpoints use Vision Transformers [24] run through the Hugging Face Transformers library [43]. An image is resized to 224 × 224 pixels and cut into $N = 196$ patches of 16 × 16 pixels; each flattened patch x_p^i is linearly projected by E, a learnable class token is prepended and positional embeddings E_{pos} are added:

$$z_0 = [x_{cls}; x_p^1 E; x_p^2 E; \ldots; x_p^N E] + E_{pos} \quad (1)$$

The sequence passes through L encoder layers of multi-head self-attention and two-layer perceptrons with residual connections, where each head computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q\,K^\top}{\sqrt{d_k}}\right)V \quad (2)$$

with query, key and value projections Q, K, V and head dimension d_k. Our inspection of the deployed checkpoints corrects an assumption made in earlier project documentation. The disease checkpoint [44] is a compact ViT with 5.5 M parameters, comparable to DeiT-Tiny [25], that predicts 13 classes: healthy and diseased conditions of corn, potato, rice and wheat plus an *invalid* class for non-leaf images; its authors report about 98% validation accuracy. The species checkpoint [45] is a ViT-Base/16 with 85.8 M parameters that distinguishes 52 medicinal and crop plants. For diagnosis, the top label is mapped to a curated treatment plan containing a severity level, immediate actions and prevention steps.

C. *Foliar Disease CNN*

To study what a small network can learn from limited data, a four-block convolutional neural network (Fig. 3) was implemented in PyTorch [46] and trained from random initialisation. Each block applies a 3 × 3 convolution, batch normalisation [47], ReLU and 2 × 2 max pooling, halving the resolution while doubling the channels from 16 to 128. The resulting 128 × 8 × 8 map is flattened, regularised with dropout [48] ($p = 0.4$) and classified by two fully connected layers. The network has 1,146,882 parameters, about five times fewer than the disease ViT and 75 times fewer than ViT-Base. It was trained for 15 epochs with Adam [49] (learning rate 10^{-3}, weight decay 10^{-4}, batch size 32) and flip and brightness augmentation [50], minimising the cross-entropy

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{c=1}^{C} y_{i,c}\log \hat{y}_{i,c} \quad (3)$$

over mini-batches of size B with $C = 2$ classes, one-hot labels y and predicted probabilities $\hat{y}$.

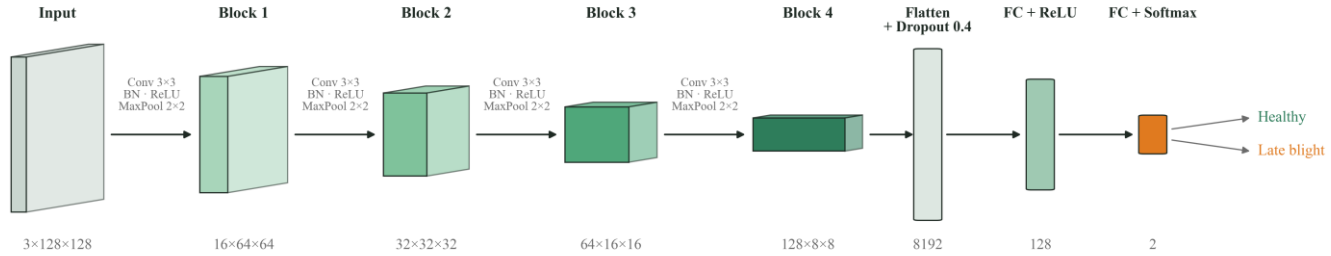


Fig. 3. Architecture of the foliar disease CNN trained from scratch. Each block halves the spatial resolution and doubles the channel depth; feature-map sizes are given as channels × height × width.

D. Tabular Learners

Crop recommender: a Random Forest [27] of 150 trees from scikit-learn [51], with depth limited to 7 and at least three samples per leaf. Each split minimises the Gini impurity

$$G(S) = 1 - \sum_{c=1}^{C} p_c^2 \tag{4}$$

of the child nodes, where p_c is the share of class c in node S, and the prediction is the majority vote of the trees. The depth limit was chosen with the sensitivity analysis in Section V-B.

Growth predictor: Ridge regression [28] on the five standardised inputs, which shrinks coefficients to reduce variance:

$$(\hat{w}, \hat{b}) = \arg\min_{w,b} \sum_{i=1}^{n} (y_i - w^\top x_i - b)^2 + \alpha \|w\|^2 \tag{5}$$

with $\alpha = 1.5$, giving only six parameters whose signs and magnitudes can be read directly by a grower.

Climate forecaster: because the series is short and the model must answer in milliseconds, forecasting is framed as supervised regression on lag features rather than with a recurrent network. A Random Forest regressor of $T = 100$ trees (depth ≤ 8, leaf size ≥ 3) averages the tree outputs f_j:

$$\hat{y}_{t+6} = \frac{1}{T} \sum_{j=1}^{T} f_j(x_t) \tag{6}$$

E. Agronomic Decision Engine

The client-side tools rest on a small physiological model, implemented in TypeScript and therefore free of any server cost. Saturation vapour pressure (kPa) at temperature T (°C) follows Tetens [32]:

$$e_s(T) = 0.61078 \exp\left(\frac{17.27\,T}{T + 237.3}\right) \tag{7}$$

and VPD is computed against a leaf that runs 1.5 °C below air temperature under transpiration [33]:

$$\mathrm{VPD} = e_s(T - 1.5) - \frac{\mathrm{RH}}{100} e_s(T) \tag{8}$$

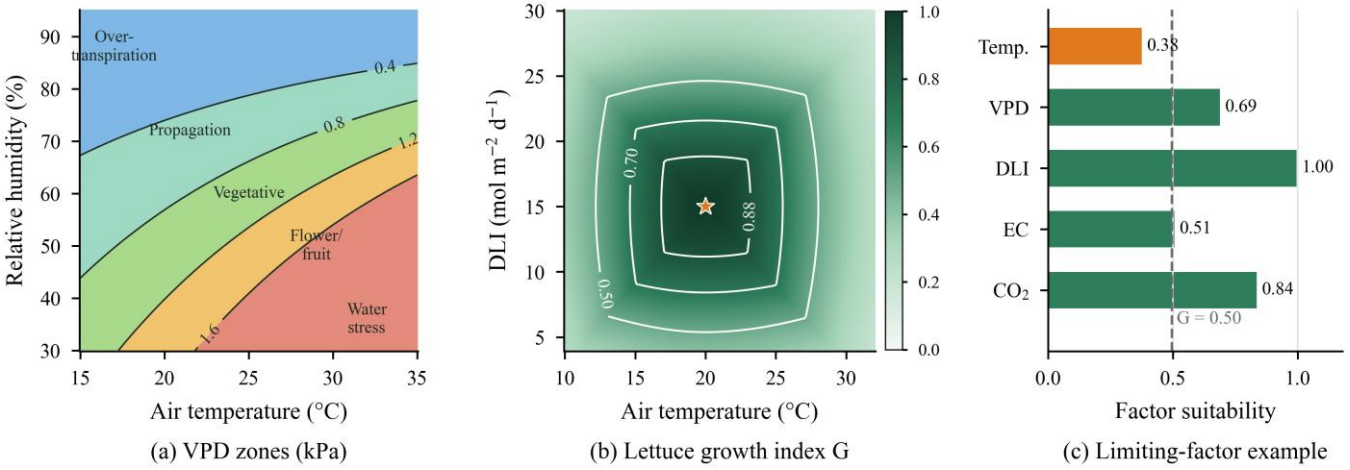
The DLI follows from the photosynthetic photon flux density (PPFD, µmol m^{-2} s^{-1}) and the photoperiod h in hours, DLI = PPFD · h · 3600 / 10^6. For each of eight crop profiles, every factor k (temperature, VPD, DLI, EC and CO_2) receives a Gaussian suitability

$$s_k = \exp\left(-\frac{1}{2}\left(\frac{v_k - v_k^*}{\sigma_k}\right)^2\right) \tag{9}$$

around its optimum v_k^* with tolerance σ_k; VPD uses a plateau over its healthy band and CO_2 a one-sided saturating score. Following Liebig's law of the minimum [34], the growth index blends the weakest factor with the geometric mean, so that several mild stresses still compound:

$$G = 0.55 \min_k s_k + 0.45 \left(\prod_{k=1}^{5} s_k\right)^{\frac{1}{5}} \tag{10}$$

The predicted cultivation time is the crop's base duration scaled by $1 + 0.8(1 - G)$, and the factor with the lowest score is reported as the limiting factor. Fig. 4 visualises the VPD zones, the growth-index surface for lettuce and a worked example.



(a) VPD zones (kPa) (b) Lettuce growth index G (c) Limiting-factor example

Fig. 4. Agronomic decision engine. (a) VPD zones over air temperature and relative humidity, from Tetens' equation with a 1.5 °C leaf offset. (b) Growth index G for butterhead lettuce over temperature and DLI (RH 65%, EC 1.2 mS/cm, CO_2 900 ppm); the star marks the optimum. (c) Factor scores for a stressed scenario (27 °C, 55% RH, DLI 14.4, EC 1.9, CO_2 600 ppm): temperature is the limiting factor and G falls to 0.50.

F. Retrieval-Augmented Advisor

The advisor follows the RAG pattern [13] (Fig. 5 and Algorithm 1). Its knowledge base holds 34 curated question–answer pairs across 12 topics, including nutrients, pH, lighting, pests, diseases, climate, water and hydroponic systems. At start-up every stored question is encoded by all-MiniLM-L6-v2 [36], [37], a six-layer, 22.7 M-parameter encoder that produces 384-dimensional L2-normalised vectors. A user question q is encoded the same way and scored against each stored question k_j by cosine similarity:

$$\mathrm{sim}(q, k_j) = \frac{q \cdot k_j}{\|q\|\|k_j\|} \tag{11}$$

The best-matching answer becomes the grounding context and is sent with a restrictive system prompt to Gemma 3 (4B) [41]. If no API key is configured, the call times out or the reply cannot be parsed, the retrieved answer is returned directly, so the advisor always produces a grounded response.

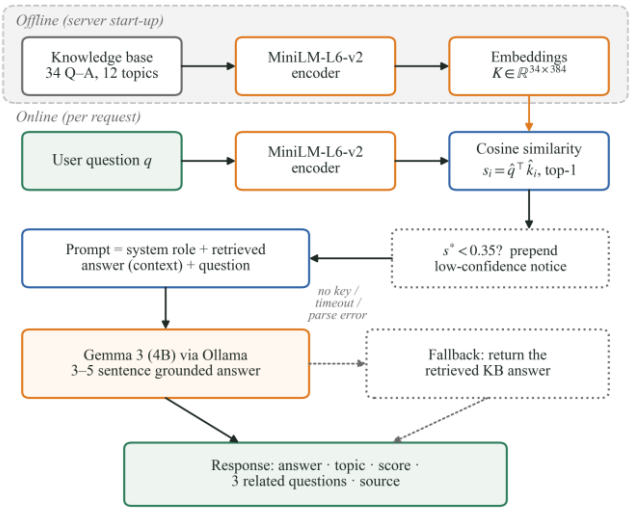


Fig. 5. Retrieval-augmented advisor. Knowledge-base questions are embedded once at start-up; each request is embedded, matched by cosine similarity and answered by Gemma 3, with a deterministic fallback to the retrieved answer.

Algorithm 1: Grounded advisory inference

Input: question q; knowledge base $\{(k_j, a_j)\}, j = 1\ldots34$; cached unit embeddings; threshold $\tau = 0.35$
Output: answer, topic, score s, related questions

$e \leftarrow \mathrm{MiniLM}(q) / \|\mathrm{MiniLM}(q)\|$
$s_j \leftarrow e \cdot \hat{k}_j$ for every j; $m \leftarrow \arg\max_j s_j$
$ctx \leftarrow a_m$
if $s_m < \tau$ **then** prepend a low-confidence notice to ctx
if an LLM key is configured **then**
  $ans \leftarrow$ Gemma3(system prompt, ctx, q)
  on timeout or parse error: $ans \leftarrow ctx$
else $ans \leftarrow ctx$
$rel \leftarrow$ up to three questions sharing the topic of entry m
return (ans, topic(m), s_m, rel)

G. Experimental Protocol

Baselines: each learned model is compared with classical alternatives trained on identical splits: for leaf images, colour-histogram features (48 histogram bins plus channel means and deviations) with logistic regression, an RBF support vector machine (SVM) [52] and a Random Forest; for crop recommendation, logistic regression, k-nearest neighbours, SVM, a decision tree, gradient boosting [53] and an unconstrained forest; for growth prediction, a mean predictor, ordinary least squares (OLS), Lasso [54], k-NN, a Random Forest and gradient boosting; and for forecasting, persistence ($\hat{y}_{t+6} = T_t$), a 24-hour seasonal-naive forecast [31], linear regression, gradient boosting and the forest without CO_2 features. Five-fold cross-validation [55] on the training split complements the single validation split.

Ablation: the CNN was retrained without augmentation, without dropout, without batch normalisation, and without both augmentation and dropout, each with three random seeds; we report mean ± standard deviation.

Retrieval benchmark: every knowledge-base entry received two paraphrased queries written to avoid the stored wording (68 queries in total). We report recall at rank k (R@k) and the mean reciprocal rank,

$$\mathrm{MRR} = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{\mathrm{rank}_i} \tag{12}$$

Metrics and hardware: classification uses accuracy, precision, recall, F1 and the area under the ROC curve [56]; regression uses MAE,

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{13}$$

and the coefficient of determination

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{14}$$

Latency is the median over repeated single-input calls after warm-up on an AMD Ryzen 7 5700U laptop CPU with 15.4 GB RAM, Python 3.13, PyTorch 2.9.1, scikit-learn 1.6.1 and Transformers 5.4. Table III lists every model with its configuration and footprint. The saved models were re-evaluated by the experiment scripts and reproduced the notebook's metrics exactly.

TABLE III. MODEL CONFIGURATIONS AND CPU FOOTPRINT

Model	Configuration	Params	Latency
Disease ViT†	ViT, 224², patch 16, 13 classes	5.5 M	26.0 ms
Species ViT†	ViT-B/16, 224², 52 classes	85.8 M	205.9 ms
MiniLM-L6-v2†	6 layers, 384-d, mean pooling	22.7 M	14.2 ms
Foliar CNN	4 conv blocks, Adam, 15 epochs	1.15 M	2.7 ms
RF recommender	150 trees, depth ≤ 7, leaf ≥ 3	12.5 k nodes	10.1 ms
Ridge growth	$\alpha = 1.5$, 5 inputs	6	0.08 ms
RF forecaster	100 trees, depth ≤ 8, leaf ≥ 3	22.8 k nodes	7.2 ms

† Pretrained model. Latency: median CPU time for one image or one query after warm-up.

V. RESULTS AND DISCUSSION

A. Foliar Disease Screening

Fig. 6 summarises the CNN. Training and validation losses fall together over the first four epochs and then settle between 0.3 and 0.4, and the two accuracy curves stay within a few points of each other, so the network does not overfit despite only 509 training images. Validation accuracy peaked at 87.5% in epoch 13 and ended at 83.59% (weighted F1 = 0.84). The confusion matrix shows that 58 of 65 healthy and 49 of 63 blighted leaves were classified correctly; most errors are missed infections, giving a late-blight recall of 0.78 against 0.89 for healthy leaves. This matches the data: about 35% of

diseased leaves carry no visible lesion. The ROC curve has an area of 0.917, so a lower decision threshold could trade some precision for higher recall on blight—often the right trade in a closed growing room.

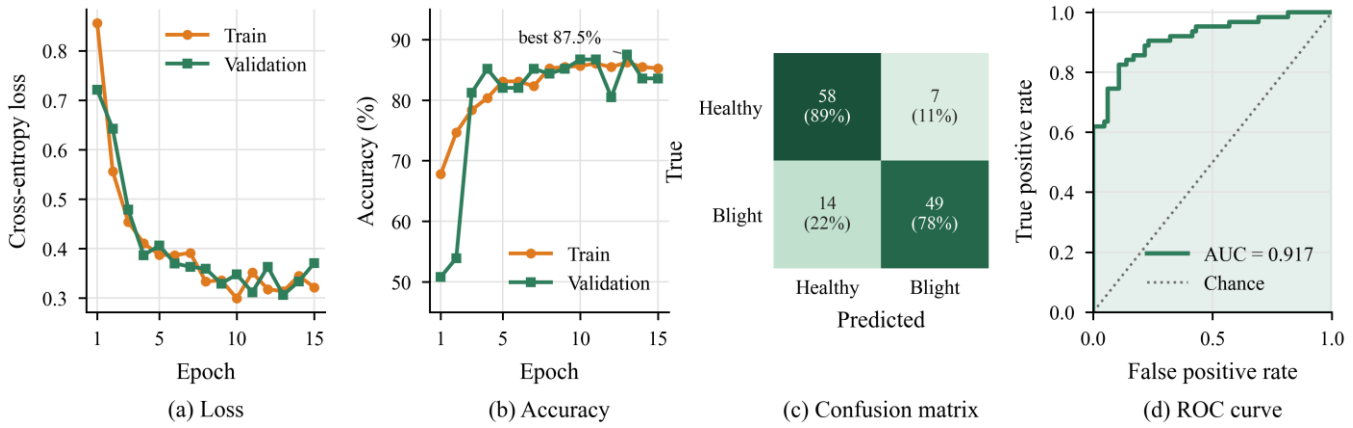


Fig. 6. Foliar disease CNN: (a) training and validation loss, (b) accuracy, (c) validation confusion matrix with counts and row percentages, and (d) ROC curve of the deployed model.

Table IV compares the CNN with classical baselines. A logistic-regression classifier on colour histograms reaches 89.84%, six points above the CNN. The ablation in Fig. 7 explains why. Removing brightness augmentation *raises* mean validation accuracy from 84.9% ± 3.3 to 87.8% ± 0.7, and removing augmentation and dropout together gives 87.5% ± 1.1, whereas removing batch normalisation lowers it to 81.8% ± 0.4. In this dataset the class signal lies mainly in global colour, and brightness jitter partly destroys it, so a model that sees raw colour statistics wins. Batch normalisation remains clearly beneficial, and dropout matters little at this scale. The lesson is that augmentation must preserve the cue that separates the classes; on real photographs, where lesion shape and texture carry the signal, the CNN's inductive bias is expected to matter much more [9], [20].

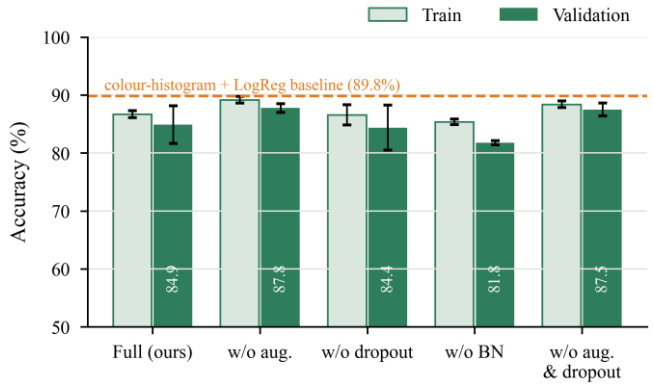


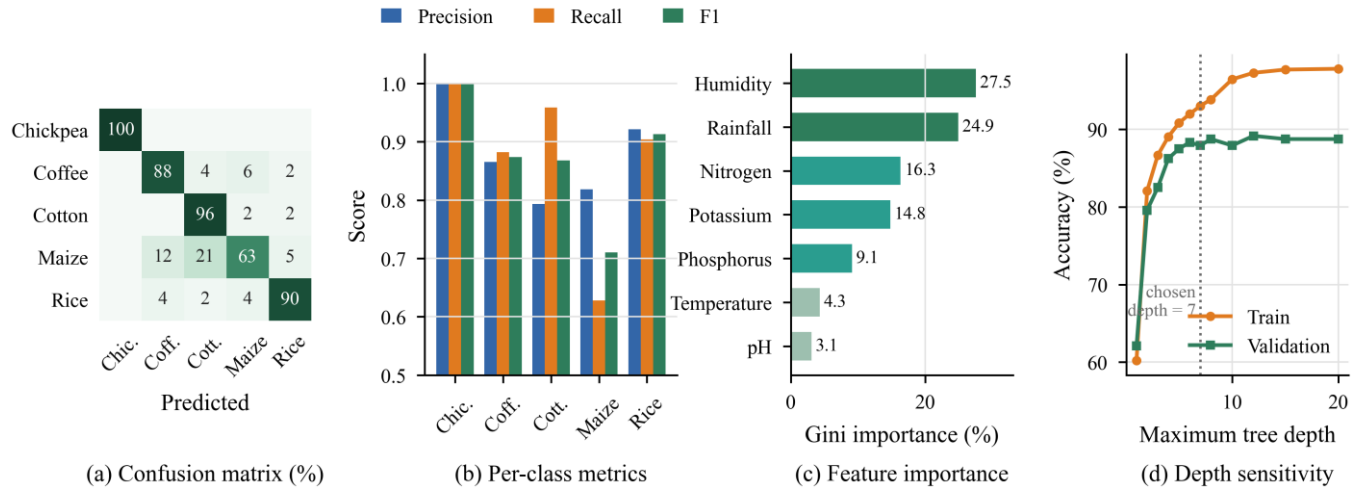
Fig. 7. CNN ablation (mean ± std over three seeds). Removing brightness augmentation helps on this colour-driven dataset, while removing batch normalisation hurts; the dashed line marks the colour-histogram baseline.

TABLE IV. CLASSIFICATION BASELINES ON THE VALIDATION SPLITS

Task / method	Acc. (%)	Macro-F1	5-fold CV (%)
Leaf disease (2 classes)			
Colour hist. + LogReg	**89.84**	**0.898**	–
Colour hist. + SVM (RBF)	88.28	0.883	–
Colour hist. + RF	**89.84**	**0.898**	–
Foliar CNN (deployed)	83.59	0.836	–
Crop recommendation (5 classes)			
Logistic regression	**91.25**	**0.910**	**89.0 ± 2.4**
SVM (RBF)	**91.25**	0.908	88.1 ± 2.6
k-NN (*k* = 5)	87.50	0.869	85.4 ± 1.6
Decision tree (depth 7)	83.75	0.831	81.3 ± 2.3
Gradient boosting	89.58	0.893	85.3 ± 2.5
RF, unconstrained	88.75	0.884	86.6 ± 2.6
RF, depth 7 (deployed)	87.92	0.873	86.6 ± 2.4

B. *Crop Recommendation*

The deployed forest reaches 93.02% training and 87.92% validation accuracy (weighted F1 = 0.876). Fig. 8 breaks this down. Chickpea is recognised perfectly because its low humidity and high potassium set it apart, while maize is hardest (recall 0.63), with 21% of maize samples predicted as cotton, whose nitrogen, temperature and humidity ranges overlap. Humidity (27.5%) and rainfall (24.9%) dominate the Gini importances, followed by nitrogen (16.3%) and potassium (14.8%), whereas temperature and pH contribute little because their ranges are similar for all five crops. The depth sweep in Fig. 8(d) shows that validation accuracy saturates near 88% from depth 6, while training accuracy keeps rising to 100% for an unconstrained forest; the chosen depth of 7 cuts the generalisation gap from 11.3 to 5.1 points at the same cross-validated accuracy (86.6%).



(a) Confusion matrix (%) (b) Per-class metrics (c) Feature importance (d) Depth sensitivity

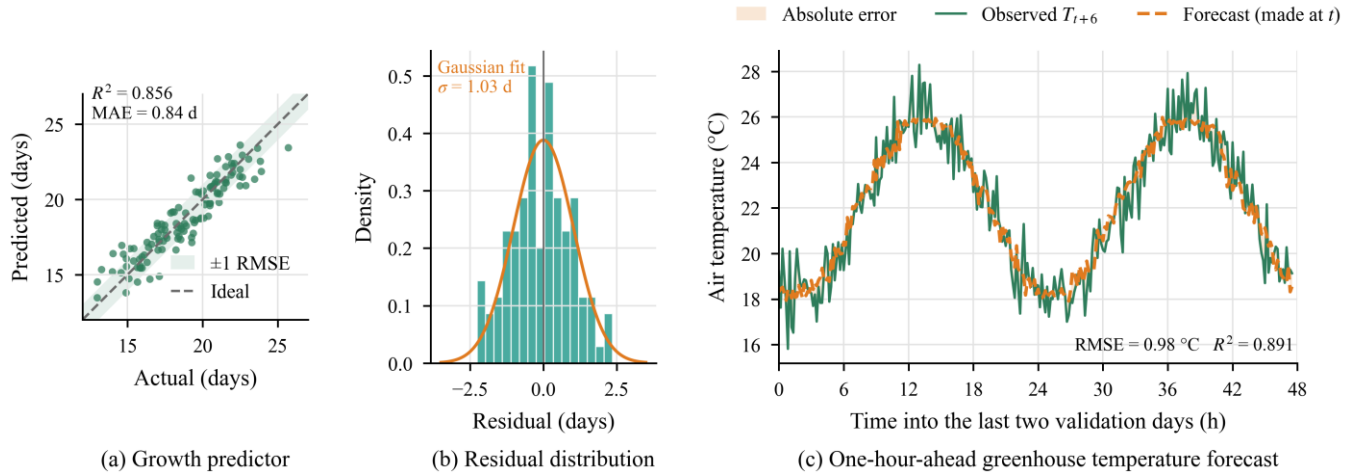
Fig. 8. Random Forest crop recommender: (a) row-normalised validation confusion matrix (%), (b) per-class precision, recall and F1, (c) Gini feature importance, and (d) train and validation accuracy as a function of maximum tree depth.

Table IV also shows that the forest is not the most accurate model: logistic regression and the SVM reach 91.25%, and logistic regression has the best cross-validated accuracy (89.0% ± 2.4). The synthetic classes are uniform boxes blurred by Gaussian noise, which linear boundaries separate well. The forest still offers per-feature explanations and robustness to unscaled or missing inputs, but on this evidence a linear model or a simple ensemble of the two would be a better default, and we report this rather than tune the comparison in our favour.

C. *Growth Prediction and Climate Forecasting*

The Ridge model explains 85.6% of the variance in growing time (MAE 0.84 days, RMSE 1.03 days). Because the noise injected into the data has σ = 1.05 days, this RMSE is essentially the irreducible error, and Fig. 9(a–b) confirms unbiased, approximately Gaussian residuals. The standardised coefficients recover the generating process: DLI (−2.11), air temperature (−0.85) and EC (−0.50) shorten the cycle, while pH (0.12) and water temperature (0.01) are correctly found to be nearly irrelevant. Table V shows that OLS and Lasso perform identically, as expected for a linear target, while flexible learners (forest, boosting, *k*-NN) do worse because they spend capacity on noise.

The climate forecaster achieves $R^2 = 0.891$, MAE 0.78 °C and RMSE 0.98 °C on the held-out final two days. It reduces RMSE by 34.2% relative to persistence and by 22.7% relative to the seasonal-naive forecast, and Fig. 9(c) shows that it tracks the diurnal cycle while smoothing sensor noise. Two observations temper this result. First, plain linear regression on the same lag features is marginally better (RMSE 0.95 °C), so the forest's non-linearity is not needed here. Second, the lagged CO_2 reading carries 79.7% of the forest's importance and removing all CO_2 features raises the RMSE to 1.15 °C: in the synthetic data CO_2 shares the daily cycle of temperature and acts as a clock. In a real greenhouse, where ventilation and enrichment decouple CO_2 from temperature, explicit time-of-day features should replace this proxy.



(a) Growth predictor (b) Residual distribution (c) One-hour-ahead greenhouse temperature forecast

Fig. 9. Regression results: (a) predicted versus actual lettuce cultivation time with ±1 RMSE band, (b) residual distribution with Gaussian fit, and (c) one-hour-ahead temperature forecast against the observed value over the last two validation days.

TABLE V. REGRESSION AND FORECASTING BASELINES

Task / method	MAE	RMSE	R^2
Lettuce growth (days)			
Mean predictor	2.29	2.73	−0.010
OLS	0.84	1.03	0.856
Lasso ($\alpha = 0.05$)	**0.83**	**1.02**	**0.858**
k-NN ($k = 5$)	1.07	1.30	0.770
Random Forest	0.91	1.13	0.828
Gradient boosting	0.93	1.13	0.827
Ridge (deployed)	0.84	1.03	0.856
Greenhouse temperature, +1 h (°C)			
Persistence	1.19	1.48	0.749
Seasonal naive (24 h)	1.02	1.26	0.818
Linear regression	0.77	**0.95**	**0.898**
Gradient boosting	**0.77**	0.96	0.894
RF without CO_2 lags	0.92	1.15	0.848
RF (deployed)	0.78	0.98	0.891

D. Advisor Retrieval

Table VI and Fig. 10 report the paraphrase benchmark. The deployed MiniLM retriever ranks the correct entry first for 79.4% of queries (MRR 0.851), against 38.2% for TF-IDF and 35.3% for BM25 on the same question index, because paraphrases such as "edges of my lettuce leaves are turning brown and crispy" share almost no words with the stored "tip burn" question. Fourteen queries are missed. Most contain a term that appears only in the stored *answer* (for example "pythium" or "substrate") or a strong lexical anchor such as a crop name. A live test reproduced this failure mode: "My lettuce roots look brown and slimy in the DWC tank" retrieved the lettuce tip-burn entry (score 0.61) instead of root rot. Indexing each question together with its answer addresses exactly this weakness and raises R@1 to 92.6% and R@3 to 98.5% at the same latency, so it will become the default. The score histogram in Fig. 10(b) shows that wrong matches concentrate below a cosine score of about 0.5, which supports the low-confidence notice at $\tau = 0.35$ and suggests that a slightly higher threshold would catch more errors.

TABLE VI. RETRIEVAL ACCURACY ON 68 PARAPHRASED QUERIES

Retriever (index)	R@1	R@3	MRR	ms
TF-IDF [38] (Q)	0.382	0.647	0.531	0.4
BM25 [39] (Q)	0.353	0.603	0.505	–
TF-IDF (Q + A)	0.544	0.779	0.688	0.4
BM25 (Q + A)	0.588	0.853	0.727	–
MiniLM (Q), deployed	0.794	0.897	0.851	14.2
MiniLM (Q + A)	**0.926**	**0.985**	**0.959**	14.8

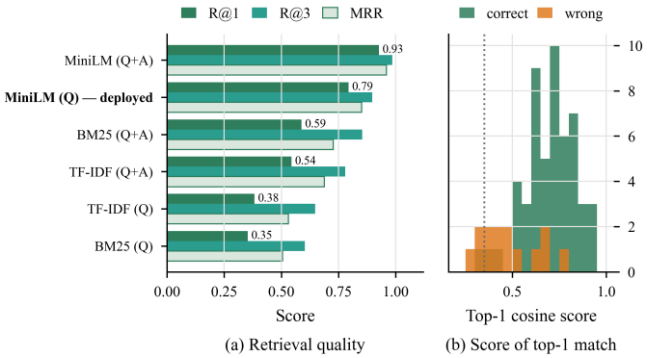


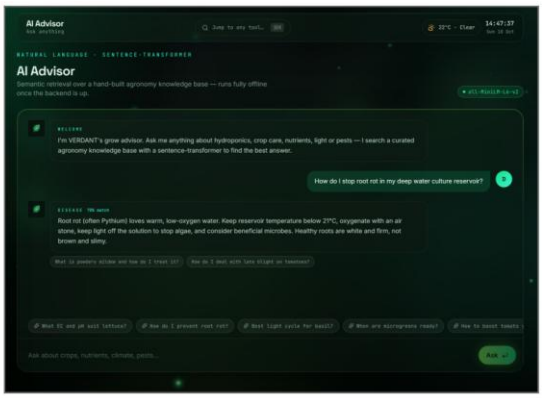
Fig. 10. Advisor retrieval: (a) R@1, R@3 and MRR for sparse and dense retrievers over question-only (Q) and question-plus-answer (Q+A) indices; (b) top-1 cosine scores of correct and incorrect matches for the deployed retriever, with the 0.35 confidence threshold.

E. Computational Footprint and System Demonstration

Table III shows that every component fits comfortably on a laptop CPU. The heaviest, the 85.8 M-parameter species ViT, answers in about 206 ms; the disease ViT needs 26 ms and a full advisor retrieval 14 ms. The four custom models together occupy under 7.7 MB on disk and respond in 0.08–10 ms, so they could run on a single-board computer next to the sensors. Fig. 11 shows the deployed interface. The Plant Doctor returns the ranked classes with a treatment plan, the advisor shows its matched topic and retrieval score, and the Grow Simulator and VPD matrix expose the agronomic engine interactively.



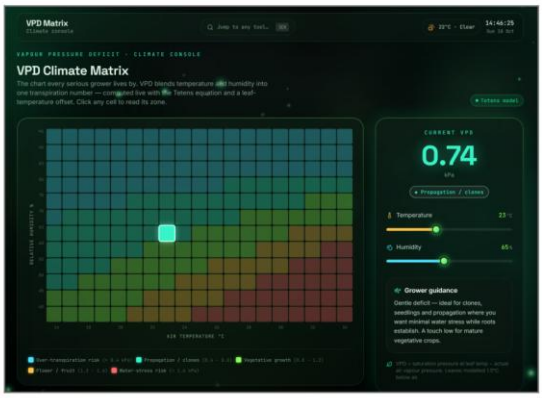
(a) Plant Doctor: ViT diagnosis with treatment plan



(b) AI Advisor: retrieval-grounded answer



(c) Grow Simulator: growth index and limiting factor



(d) VPD Matrix: climate zone console

Fig. 11. The deployed VERDANT interface (captured from the running system): (a) Plant Doctor diagnosing a potato leaf with the disease ViT, (b) AI Advisor answering a root-rot question through retrieval, (c) Grow Simulator showing the growth index and limiting factor, and (d) the VPD climate matrix.

F. *Limitations and Threats to Validity*

Three limitations bound these conclusions. *External validity:* the four custom models were trained and evaluated on synthetic data. Their scores demonstrate a sound pipeline, not field performance, and the CNN results in particular show how a synthetic generator can reward shortcuts (here, global colour) that real images would not; retraining on photographs and sensor logs from a working farm is a precondition for deployment. *Pretrained components:* the accuracy figures for the ViT checkpoints are those reported by their authors and were not re-measured on vertical-farm imagery, whose lighting and backgrounds differ from public datasets [9], [22]; moreover, the disease ViT does not cover tomato, which is why the tomato-specific CNN was developed. *Advisor scope:* the knowledge base is small, the retrieval benchmark was written by the project team, and the quality of generated answers was not scored against a formal benchmark. Finally, the custom models are currently evaluated offline and are not yet exposed through the REST API.

VI. Conclusion and Future Work

This paper presented VERDANT, a CPU-deployable platform that helps urban growers diagnose plant diseases, identify species, plan crops, estimate harvest dates, reason about climate and obtain grounded agronomic advice from one interface. Beyond integrating these functions, we evaluated every learned component against classical baselines and with ablation and sensitivity studies. Dense retrieval more than doubles the accuracy of keyword search for paraphrased questions, and a one-line change to the index raises it further to 92.6%. The forecaster cuts persistence error by a third. The baselines also show that simple models match or beat more complex ones on the current synthetic data.

Future work will (i) collect real leaf images and sensor logs from a partner vertical farm to retrain and validate the custom models; (ii) expose the custom models through the API and connect live IoT sensors so the forecaster can raise alerts and drive actuators; (iii) replace the CO_2 proxy with explicit time features and compare against sequence models [30] on longer series; (iv) switch the advisor to question-plus-answer indexing, enlarge the knowledge base and evaluate generated answers with a formal benchmark; and (v) quantise the vision models for on-device inference in a mobile application.

Acknowledgment

The authors thank their project guide, [Guide Name], and the School of Engineering, Anurag University, for their guidance and for providing the facilities needed to carry out this work.

References

[1] United Nations, Department of Economic and Social Affairs, Population Division, "World urbanization prospects: The 2018 revision," United Nations, New York, NY, USA, Rep. ST/ESA/SER.A/420, 2019.

[2] Food and Agriculture Organization of the United Nations, The Future of Food and Agriculture—Trends and Challenges. Rome, Italy: FAO, 2017.

[3] D. Despommier, The Vertical Farm: Feeding the World in the 21st Century. New York, NY, USA: Thomas Dunne Books, 2010.

[4] T. Kozai, G. Niu, and M. Takagaki, Eds., Plant Factory: An Indoor Vertical Farming System for Efficient Quality Food Production, 2nd ed. London, U.K.: Academic Press, 2019.

[5] K. Benke and B. Tomkins, “Future food-production systems: Vertical farming and controlled-environment agriculture,” Sustainability: Sci., Pract. Policy, vol. 13, no. 1, pp. 13–26, 2017.

[6] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, “The global burden of pathogens and pests on major food crops,” Nature Ecol. Evol., vol. 3, no. 3, pp. 430–439, 2019.

[7] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, “Machine learning in agriculture: A review,” Sensors, vol. 18, no. 8, Art. no. 2674, 2018.

[8] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” Comput. Electron. Agric., vol. 147, pp. 70–90, 2018.

[9] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” Front. Plant Sci., vol. 7, Art. no. 1419, 2016.

[10] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” Comput. Electron. Agric., vol. 145, pp. 311–318, 2018.

[11] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, “Big data in smart farming—A review,” Agric. Syst., vol. 153, pp. 69–80, 2017.

[12] O. Elijah, T. A. Rahman, I. Orikumhi, C. Y. Leow, and M. N. Hindia, “An overview of Internet of Things (IoT) and data analytics in agriculture: Benefits and challenges,” IEEE Internet Things J., vol. 5, no. 5, pp. 3758–3773, Oct. 2018.

[13] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 9459–9474.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2012, pp. 1097–1105.

[15] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2009, pp. 248–255.

[16] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[17] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015.

[18] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, “Deep neural networks based recognition of plant diseases by leaf image classification,” Comput. Intell. Neurosci., vol. 2016, Art. no. 3289801, 2016.

[19] M. Brahimi, K. Boukhalfa, and A. Moussaoui, “Deep learning for tomato diseases: Classification and symptoms visualization,” Appl. Artif. Intell., vol. 31, no. 4, pp. 299–315, 2017.

[20] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, “A comparative study of fine-tuning deep learning models for plant disease identification,” Comput. Electron. Agric., vol. 161, pp. 272–279, 2019.

[21] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778.

[22] R. Thapa, K. Zhang, N. Snavely, S. Belongie, and A. Khan, “The Plant Pathology Challenge 2020 data set to classify foliar disease of apples,” Appl. Plant Sci., vol. 8, no. 9, Art. no. e11390, 2020.

[23] A. Vaswani et al., “Attention is all you need,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 5998–6008.

[24] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.

[25] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in Proc. Int. Conf. Mach. Learn. (ICML), 2021, pp. 10347–10357.

[26] Y. Borhani, J. Khoramdel, and E. Najafi, “A deep learning based approach for automated plant disease classification using vision transformer,” Sci. Rep., vol. 12, Art. no. 11554, 2022.

[27] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[28] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” Technometrics, vol. 12, no. 1, pp. 55–67, 1970.

[29] R. R. Shamshiri et al., “Advances in greenhouse automation and controlled environment agriculture: A transition to plant factories and urban agriculture,” Int. J. Agric. Biol. Eng., vol. 11, no. 1, pp. 1–22, 2018.

[30] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.

[31] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, VIC, Australia: OTexts, 2021.

[32] O. Tetens, “Über einige meteorologische Begriffe,” Z. Geophys., vol. 6, pp. 297–309, 1930.

[33] J. L. Monteith and M. H. Unsworth, Principles of Environmental Physics, 4th ed. Oxford, U.K.: Academic Press, 2013.

[34] J. von Liebig, Die organische Chemie in ihrer Anwendung auf Agricultur und Physiologie. Braunschweig, Germany: Vieweg, 1840.

[35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL-HLT), 2019, pp. 4171–4186.

[36] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP-IJCNLP), 2019, pp. 3982–3992.

[37] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 5776–5788.

[38] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Inf. Process. Manage., vol. 24, no. 5, pp. 513–523, 1988.

[39] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.

[40] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2020, pp. 6769–6781.

[41] Gemma Team, “Gemma 3 technical report,” arXiv:2503.19786, 2025.

[42] S. I. Nikolenko, Synthetic Data for Deep Learning. Cham, Switzerland: Springer, 2021.

[43] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations (EMNLP), 2020, pp. 38–45.

[44] wambugu71, “crop_leaf_diseases_vit,” Hugging Face Model Hub. [Online]. Available: https://huggingface.co/wambugu71/crop_leaf_diseases_vit

[45] dima806, “medicinal_plants_image_detection,” Hugging Face Model Hub. [Online]. Available: https://huggingface.co/dima806/medicinal_plants_image_detection

[46] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019, pp. 8024–8035.

[47] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 448–456.

[48] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” J. Mach. Learn. Res., vol. 15, no. 56, pp. 1929–1958, 2014.

[49] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[50] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” J. Big Data, vol. 6, Art. no. 60, 2019.

[51] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[52] C. Cortes and V. Vapnik, “Support-vector networks,” Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995.

[53] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[54] R. Tibshirani, “Regression shrinkage and selection via the lasso,” J. Roy. Statist. Soc. B, vol. 58, no. 1, pp. 267–288, 1996.

[55] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in Proc. Int. Joint Conf. Artif. Intell. (IJCAI), 1995, pp. 1137–1143.

[56] T. Fawcett, “An introduction to ROC analysis,” Pattern Recognit. Lett., vol. 27, no. 8, pp. 861–874, 2006.